

Plano de Testes - API e WEB

Plano de Testes - Cinema App

1. Apresentação

O planejamento de testes em questão descreve os meios e estratégias a serem seguidas para validar a qualidade do back-end e front-end da aplicação Cinema App, a qual simula um sistema de reserva de ingressos para um cinema.

A versão da aplicação testada é a 1.0.0, a qual corresponde à versão mais atual.

2. Objetivo

Identificar falhas funcionais na aplicação Cinema App por meio de testes funcionais realizados com Robot Framework, utilizando diferentes técnicas para guiar a execução dos testes, a fim de propor melhorias para as funcionalidades testadas e apontar defeitos que possam comprometer a qualidade dos serviços.

3. Escopo

Em um primeiro momento, serão aplicados testes exploratórios freestyle a fim de compreender melhor o sistema e adquirir uma maior confiança na criação e execução de futuros testes automatizados. Os testes exploratórios serão executados manualmente no front-end da aplicação, enquanto no back-end os testes exploratórios serão feitos utilizando a ferramenta Postman.

Ademais, serão aplicados testes funcionais em todas as funcionalidades do sistema, abrangendo testes de API e testes WEB de todos os fluxos principais e fluxos alternativos de cenários considerados críticos, ou seja, aqueles em que a prioridade mais alta for atribuída após a análise de riscos.

Não serão realizados testes não-funcionais.

4. Pessoas envolvidas

Papel	Nome	Responsabilidade
Testador	Lívia	Executa e documenta os testes realizados na aplicação.

5. Análise

A aplicação submetida a teste simula um sistema de reservas de ingressos para um cinema, a qual oferece ao usuário uma interface para visualizar filmes, consultar sessões e reservar assentos, além de funcionalidades para o administrador de gerenciamento de filmes, sessões e reservas. Vale deixar claro que todas as funcionalidade estão registradas na documentação da API do sistema, a qual é uma aplicação RESTful e possui uma arquitetura baseada em endpoints que utilizam os métodos HTTP padrão (POST, GET, PUT e DELETE). Esse produto exige que as APIs sejam bem definidas e que o front-end e o back-end estejam em sincronia, com persistência no banco de dados e controle de acesso entre perfis de usuário regular, visitante e administrador.

No que tange aos riscos do produto, destacam-se vulnerabilidades de segurança - com possível exposição de dados do usuário e controles de acesso aos endpoints mal protegidos -, e inconsistências na reserva - visto que, de acordo com uma análise prévia da aplicação, duas pessoas podem reservar o mesmo assento simultaneamente.

6. Técnicas de teste utilizadas

Serão aplicadas as seguintes técnicas em todos os módulos da aplicação:

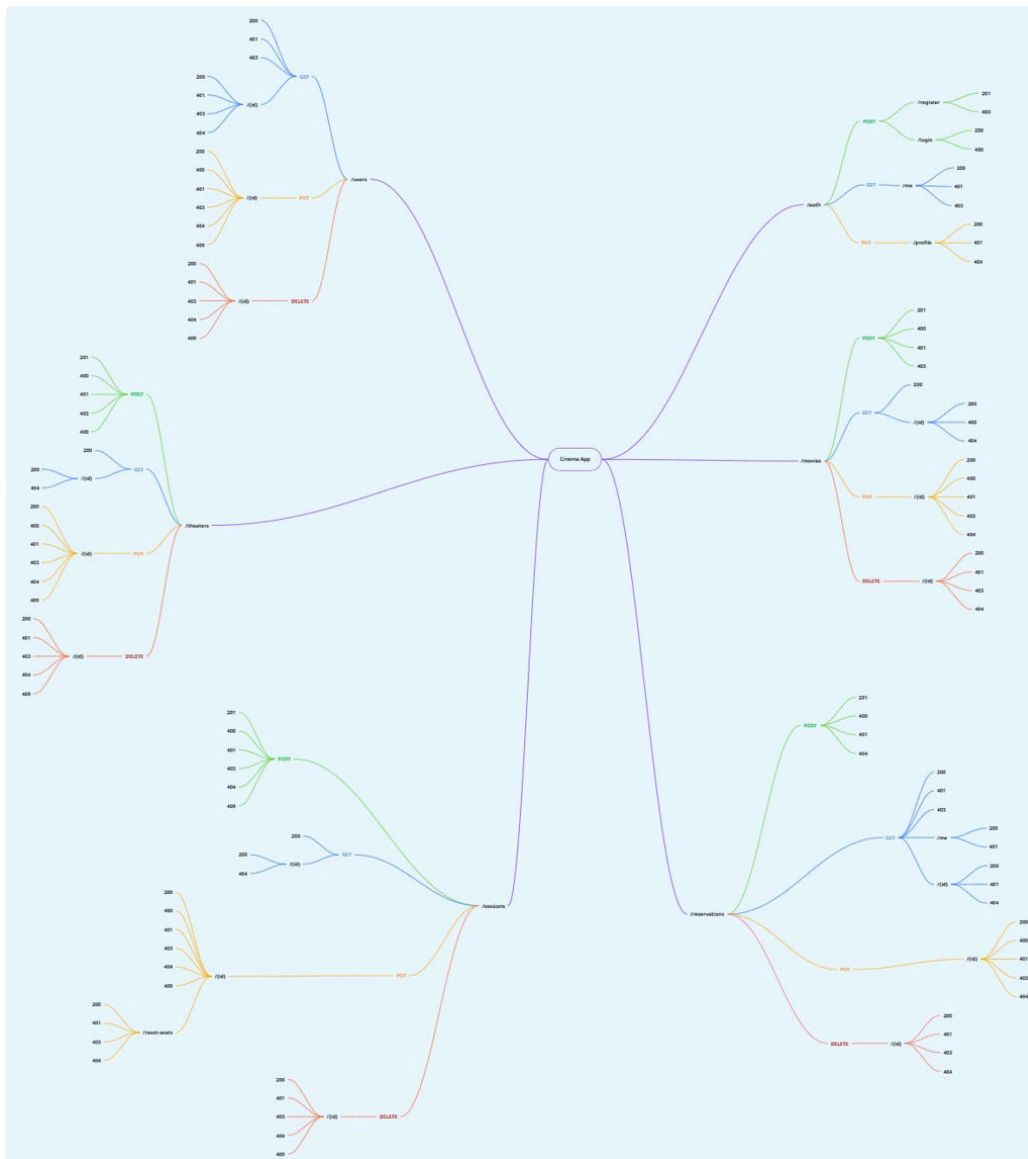
- Partição de Equivalência, para validar diferentes grupos de entradas válidas e inválidas nos campos;
- Null, Zero, Vazio, para verificar o comportamento da aplicação em cenários de dados obrigatórios nulos ou em branco;
- Tabela de Decisão, para validar diferentes combinações de entradas ao realizar o login.

Todas as técnicas serão apoiadas pela Heurística CRUD, uma vez que esta garante a cobertura de cenários de criação, leitura, atualização e exclusão de dados.

7. Mapa mental da aplicação

Para compreender os serviços da aplicação foi desenvolvido um mapa mental da API com todos os endpoints incluídos, bem como um mapa mental do front-end da aplicação.

7.1. Mapa mental da API



7.2. Mapa mental do front-end

8. Recursos necessários

- Máquina com acesso à internet;
- Navegador Google Chrome;
- Instalação do Node.js;
- Instalação e configuração do MongoDB (se utilizada a opção local);
- Instalação do back-end da aplicação por meio do seguinte link: [GitHub - juniorschmitz/cinema-challenge-back: Back-end for the QE PB Final Challenge.](https://github.com/juniorschmitz/cinema-challenge-back)
- Instalação do front-end da aplicação por meio do seguinte link: [GitHub - juniorschmitz/cinema-challenge-front: Front-end for the QE PB Final Challenge.](https://github.com/juniorschmitz/cinema-challenge-front)
- Ferramenta Postman para a execução de testes exploratórios de API.

- Plataforma para reportar os bugs e melhorias (Jira);
- Configuração da ferramenta QAlity no projeto Jira;
- Ferramenta para criar e executar os testes (Robot Framework);

9. Critérios usados

• Critérios de Início:

- Back-end e Front-end do Cinema App instalado localmente;
- Criação dos cenários de teste;
- Criação dos scripts de testes automatizados com Robot Framework.

• Estratégia de Teste:

- Baseada em Riscos.

• Critérios de Conclusão:

- Todos os cenários de teste com prioridade alta executados;
- Relatório de issues e melhorias finalizado.

10. Cenários de teste

Todos os cenários de teste podem ser acessados por meio do seguinte documento: [Cenários de Teste](#) .

11. Priorização da execução dos cenários

Os cenários de teste foram priorizados de acordo com o nível de risco das funcionalidades, sendo consideradas de alta prioridade cenários de teste com nível de risco de 15 a 25, de média prioridade cenários de teste com nível de risco de 5 a 15, e de baixa prioridade cenários de teste com nível de risco de 0 a 5.

Vale ressaltar que, devido a sua grande importância para o funcionamento da aplicação, todos os cenários com fluxos comuns receberam alta prioridade.

Por fim, o resultado da priorização dos cenários de teste foi o seguinte:

Alta prioridade: Cenários do módulo Authentication →; Cenários do módulo Movies →; Cenários do módulo Reservations →; Cenários do módulo Sessions →; Cenários do módulo Theaters →; Cenários do módulo Users →.

Média prioridade: Cenários do módulo Authentication →; Cenários do módulo Movies →; Cenários do módulo Reservations →; Cenários do módulo Sessions →; Cenários do módulo Theaters →; Cenários do módulo Users →.

Baixa prioridade: Cenários do módulo Authentication →; Cenários do módulo Movies →; Cenários do módulo Reservations →; Cenários do módulo Sessions →; Cenários do módulo

Theaters → ; Cenários do módulo Users → .

12. Matriz de risco

Desenvolveu-se uma matriz de risco com base em todos os possíveis riscos das funcionalidades do sistema, onde:

- **Pontuação de Probabilidade:** de 1 a 5, onde:

- 1 = Muito baixa;
- 2 = Baixa;
- 3 = Média;
- 4 = Alta;
- 5 = Muito alta.

- **Pontuação de Impacto:** de 1 a 5, onde:

- 1 = Muito baixo;
- 2 = Baixo;
- 3 = Médio;
- 4 = Alto;
- 5 = Muito alto.

- **Pontuação de Classificação:** resultado da multiplicação entre a pontuação de probabilidade e a pontuação de risco.

A Matriz de Risco pode ser acessada pelo link a seguir: [📄 Matriz de Risco](#)

13. Cobertura dos testes

13.1. Cobertura de Entrada

13.2. Cobertura de Saída

14. Testes candidatos à automação

15. Como os resultados de teste serão divulgados

Os resultados serão divulgados por intermédio de um repositório no GitHub, contendo o plano de testes desenvolvido, os cenários de teste mapeados, os scripts de teste desenvolvidos com Robot Framework, além de um documento com o mapeamento de issues e melhorias apontadas com base nos testes realizados. Ademais, o planejamento e os resultados serão transmitidos por meio de uma apresentação verbal em uma reunião com a equipe.

16. Cronograma

[illegible]

de issu es e melh oria s											
Ap re sent ação dos resu ltad os											

17. Mapeamento de Issues e Melhorias